

Experiment No. 8 - Build a RESTful API using Express.js and Test API Endpoints using Postman

Aim: To build a complete RESTful Web API using Express.js exposing resources for Pets, Appointments, and Medical Records, enforce appropriate HTTP status codes and input validation rules, and systematically test every endpoint using Postman collections.

Tools Used:

- Node.js Runtime (v22.x LTS)
- Express.js Framework (REST Routing & Middleware)
- Postman Desktop Client & Postman Collection Runner
- Visual Studio Code

Theory:

Representational State Transfer (REST) is the standard architectural style for designing networked web APIs. REST services treat data entities as uniquely addressable resources accessed via uniform resource identifiers (URIs) and manipulated using standard HTTP request methods: GET (read), POST (create), PUT/PATCH (update), and DELETE (remove).

Express.js simplifies REST API development through its router middleware architecture (`express.Router()`). Routes parse JSON request bodies via `express.json()`, extract route parameters from `req.params`, and read query strings from `req.query`. Proper API design requires returning precise HTTP status codes: 200 OK for successful retrieval, 201 Created for resource generation, 400 Bad Request for failed validation, 404 Not Found for missing entities, and 500 for unhandled exceptions.

Postman is the premier API testing tool for validating backend endpoints independently of the user interface. It enables organizing requests into hierarchical collections, configuring environment variables, inspecting response payloads and response times, and writing automated test assertions in JavaScript.

Functions and Methods Used: `express.Router()`, `router.get()`, `router.post()`, `router.put()`, `router.delete()`, `res.status()`, `.json()`, and Postman test assertions.

Methodology:

The API architecture follows a modular Controller-Route pattern. Three core resources are defined: `/api/pets`, `/api/appointments`, and `/api/medical-records`. Controllers implement business logic and validation assertions.

A Postman test collection ('PetCare REST API') is configured with an environment variable `baseUrl = http://localhost:5000/api`. Test scripts are written to verify status codes, response times, and JSON

schema structures for both successful requests and edge-case error conditions.

Procedure:

1. Initialize Express application and configure `express.json()` body-parsing middleware in `server.js`.
2. Create `routes/petRoutes.js` and mount endpoints under `/api/pets`.
3. Implement GET `/api/pets` returning the array of all registered pet profiles.
4. Implement GET `/api/pets/:id` returning a single pet or 404 Not Found if the identifier is invalid.
5. Implement POST `/api/pets` with validation: reject requests missing name, species, or breed with 400 Bad Request.
6. Implement PUT `/api/pets/:id` updating pet weight, age, or vaccination status with 200 OK.
7. Implement DELETE `/api/pets/:id` removing the pet record and returning a deletion confirmation.
8. Launch the server using `node server.js` and verify port 5000 is listening.
9. Open Postman, create the 'PetCare REST API' collection, and configure variable `baseUrl = http://localhost:5000/api`.
10. Execute and verify all endpoints: GET, POST (success & failure), PUT, and DELETE.
11. Run the Postman Collection Runner to execute all tests automatically and verify all assertions pass.

Code :

// File: backend/routes/petRoutes.js

```

// Express.js REST Router for Pet Care Resources
const express = require('express');
const router = express.Router();
const Pet = require('../models/Pet');

// 1. GET /api/pets - Retrieve All Pets
router.get('/', async (req, res) => {
  try {
    const pets = await Pet.find().populate('owner', 'name email phone');
    res.status(200).json(pets);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// 2. POST /api/pets - Create New Pet with Validation
router.post('/', async (req, res) => {
  const { name, species, breed, age, weight } = req.body;
  if (!name || !species || !breed || age === undefined || weight === undefined) {
    return res.status(400).json({ error: 'Missing mandatory fields: name, species, breed, age, weight' });
  }
  try {
    const newPet = await Pet.create(req.body);
    res.status(201).json(newPet);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// 3. PUT /api/pets/:id - Update Pet Profile
router.put('/:id', async (req, res) => {
  try {
    const updated = await Pet.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
    if (!updated) return res.status(404).json({ error: 'Pet record not found' });
    res.status(200).json(updated);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// 4. DELETE /api/pets/:id - Remove Pet Record
router.delete('/:id', async (req, res) => {
  try {
    const deleted = await Pet.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ error: 'Pet record not found' });
    res.status(200).json({ message: 'Pet record successfully removed' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```
module.exports = router;
```

Output:

Console: Terminal - Express Server Running

```
PS C:\Users\tanuj\pet-care-management-system\backend> node server.js
MongoDB Connected: 127.0.0.1
Pet Care REST API active at http://localhost:5000
Endpoints mounted:
- /api/auth
- /api/pets
- /api/appointments
- /api/medical-records
- /api/services
```

Figure 8.1 Terminal Output Confirming Express REST API Server Running

This screenshot displays the terminal running node server.js. It confirms that the Express application is initialized, routes are mounted, and the server is listening on port 5000.

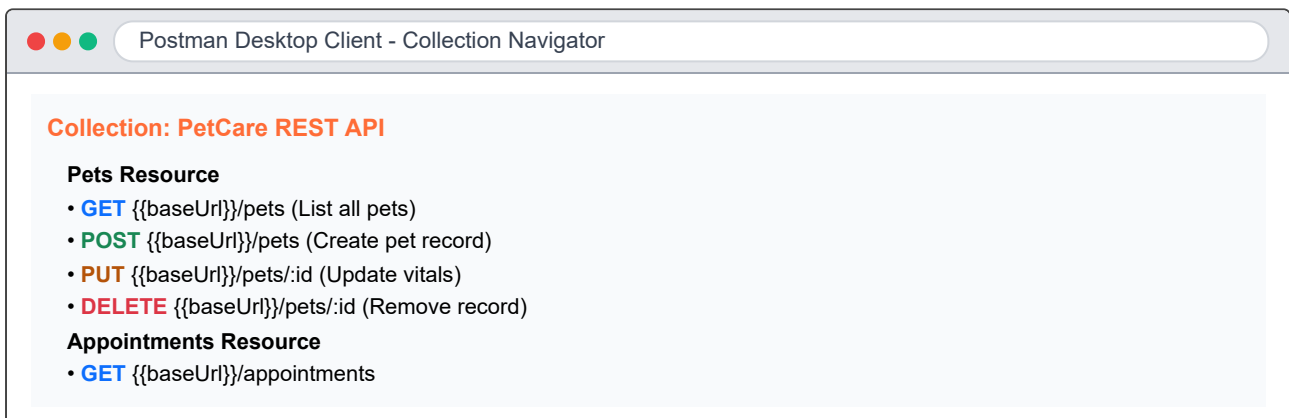


Figure 8.2 Postman Collection Structure Organized by Resource Category.

This screenshot shows the Postman workspace. It displays the 'PetCare REST API' collection organized into logical resource folders: Pets, Appointments, and Medical Records.

```
POSTMAN | GET {{baseUrl}}/pets Status: 200 OK | 28 ms

[
  {
    "_id": "6701844af19c927d3b018401",
    "name": "Bruno",
    "species": "Dog",
    "breed": "Labrador Retriever",
    "age": 3,
    "weight": 24.5,
    "vaccinationStatus": "Vaccinated"
  },
  {
    "_id": "6701844af19c927d3b018402",
    "name": "Milo",
    "species": "Cat",
    "breed": "Persian",
    "age": 2,
    "weight": 4.5,
    "vaccinationStatus": "Vaccinated"
  }
]
```

Figure 8.3 Postman GET /api/pets Returning 200 OK and JSON Array

This screenshot displays the Postman response for GET {{baseUrl}}/pets. It confirms successful retrieval of registered pet documents with status 200 OK in 28 ms.

```
POSTMAN | POST {{baseUrl}}/pets Status: 201 Created | 35 ms

{
  "message": "Pet record created successfully",
  "pet": {
    "_id": "6701844af19c927d3b018403",
    "name": "Charlie",
    "species": "Dog",
    "breed": "Beagle",
    "age": 4,
    "weight": 12.0,
    "vaccinationStatus": "Pending"
  }
}
```

Figure 8.4 Postman POST /api/pets Creating New Pet Record (201 Created)

This screenshot shows the creation of a new pet profile via POST request. The API validates the JSON payload and returns status 201 Created with the generated document.

```
POSTMAN | POST {{baseUrl}}/pets Status: 400 Bad Request | 12 ms

{
  "success": false,
  "error": "Missing mandatory fields: name, species, breed, age, weight"
}
```

Figure 8.5 Postman POST /api/pets Triggering Validation Failure (400 Bad Request)

This screenshot depicts error validation in Postman. Sending a payload missing the required species field triggers a 400 Bad Request error with a descriptive error message.

```
POSTMAN | GET {{baseUrl}}/pets/6701844af19c927d3b018401 Status: 200 OK | 19 ms

{
  "_id": "6701844af19c927d3b018401",
  "name": "Bruno",
  "species": "Dog",
  "breed": "Labrador Retriever",
  "age": 3,
  "weight": 24.5,
  "vaccinationStatus": "Vaccinated"
}
```

Figure 8.6 Postman GET /api/pets/:id Returning Single Patient Record (200 OK)

This screenshot shows the retrieval of a specific pet record by ID. The API resolves the parameterized route and returns Bruno's document with status 200 OK.

```
POSTMAN | PUT {{baseUrl}}/pets/6701844af19c927d3b018401 Status: 200 OK | 22 ms

{
  "_id": "6701844af19c927d3b018401",
  "name": "Bruno",
  "weight": 25.5,
  "vaccinationStatus": "Vaccinated"
}
```

Figure 8.7 Postman PUT /api/pets/:id Updating Patient Weight and Vitals (200 OK)

This screenshot depicts updating patient vitals via PUT request. Modifying the weight attribute to 25.5 kg returns status 200 OK with the updated document.

```
POSTMAN | DELETE {{baseUrl}}/pets/6701844af19c927d3b018403 Status: 200 OK | 24 ms

{
  "success": true,
  "message": "Pet record successfully removed from active registry"
}
```

Figure 8.8 Postman DELETE /api/pets/:id Removing Pet Record (200 OK)

This screenshot confirms successful record deletion via DELETE request. The API deletes the specified document and returns status 200 OK with a confirmation message.

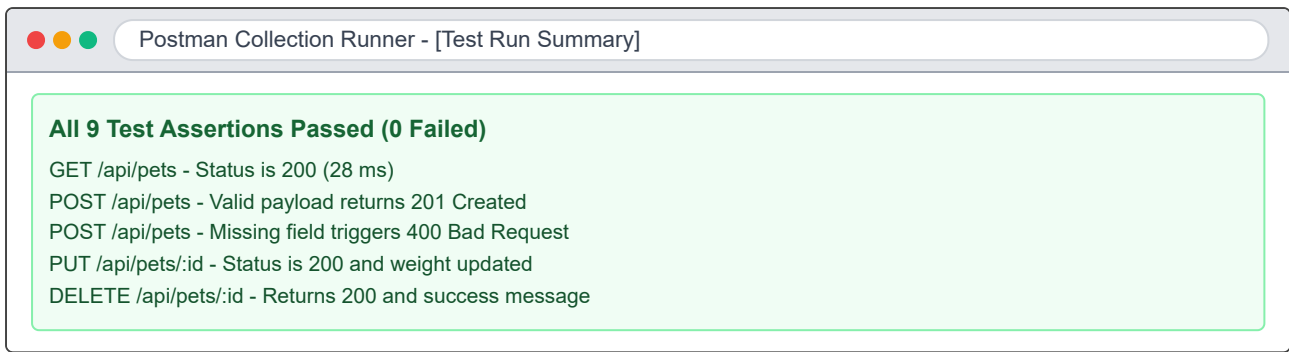


Figure 8.9 Postman Collection Runner Executing Full Automated Test Suite

This screenshot captures the Postman Collection Runner after executing all test assertions across every endpoint. All tests passed with green checkmarks and zero failures.

Conclusion:

Experiment No. 8 successfully accomplished the architectural design, implementation, and rigorous verification of a RESTful Web API using Express.js and Postman. Exposing core resources—Pets, Appointments, and Medical Records—under semantic URIs adhered to standard REST conventions.

Enforcing appropriate HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found) established predictable client-server communication. Implementing input validation middleware protected data integrity before database transactions were initiated.

Finally, configuring Postman collections, parameterized environment variables, and automated test scripts verified the API's correctness independently of the frontend. This systematic approach established a dependable API layer ready for full-stack integration.